Escalado de microservicios: Kubernetes y serverless

El requisito previo: servicios sin estado

Un servicio sin estado (stateless) es aquel que no guarda en memoria local nada que se necesite para atender el pedido siguiente. Cada instancia puede responder cualquier pedido sin depender de haber atendido los anteriores. Esta es la condicion que habilita el escalado horizontal: si el servicio no guarda estado propio, agregar instancias es simplemente agregar capacidad.

El estado de sesion se saca del proceso y se lleva a un almacen compartido, tipicamente un cache distribuido como Redis, o se codifica en un token firmado que viaja con cada pedido. Los archivos subidos por el usuario van a un almacenamiento de objetos, no al disco local del contenedor, porque ese disco desaparece cuando la instancia se recrea.

Escalado horizontal en Kubernetes

Kubernetes escala replicas de un Deployment mediante el HorizontalPodAutoscaler. El HPA observa una metrica, tipicamente uso de CPU o memoria, y compara contra un valor objetivo. Si el promedio de las replicas supera el objetivo, crea replicas nuevas; si queda por debajo, las reduce.

Para que el HPA funcione con sentido hay que declarar requests y limits de recursos en el contenedor. El request es lo que el planificador reserva para ubicar el pod en un nodo, y es la base sobre la que se calcula el porcentaje de uso. Sin requests declarados el HPA no tiene denominador y no puede decidir.

El escalado en Kubernetes no es instantaneo. Entre que la metrica sube, el HPA la observa, se crea el pod, arranca la JVM y la sonda de readiness pasa a verde, transcurren decenas de segundos. Por eso conviene dejar un margen de capacidad por encima del uso normal en vez de ajustar al limite.

Serverless y el arranque en frio

En un modelo serverless la plataforma crea instancias por pedido y las destruye cuando no hay trafico, y la facturacion es por ejecucion. Cuando llega un pedido y no hay instancia disponible, la plataforma tiene que crear una desde cero: eso es el arranque en frio, y en aplicaciones sobre JVM es especialmente caro porque hay que levantar la maquina virtual e inicializar el contexto de la aplicacion.

Cuando conviene cada uno

Con trafico constante y predecible, Kubernetes es la mejor opcion. Las instancias ya estan levantadas y tibias, no hay arranque en frio, y el costo por unidad de computo reservada es menor que el costo por ejecucion. Serverless conviene con trafico intermitente o con picos muy marcados y valles largos, donde pagar por instancias ociosas seria el gasto dominante.

La regla practica es mirar el factor de utilizacion. Si las instancias estarian ocupadas la mayor parte del tiempo, conviene reservarlas. Si estarian ociosas la mayor parte del tiempo, conviene pagar por ejecucion.